Frequent Itemset Mining with FP-Growth

ESE 327 Project Report

Nusy Islam

October 30, 2026

Abstract

I implemented the FP-Growth frequent itemset mining algorithm in C++17, verified it against the Han and Kamber textbook example, a brute-force reference, and the mlxtend library, and measured it on four UCI datasets over a wide range of support thresholds. I also built an OpenMP version and a CUDA hybrid version and compared all three on a desktop with an Intel i7-14700K and an RTX 4070 GPU, and on an NVIDIA Jetson Orin Nano in the lab. On the Jetson's six cores OpenMP gave best speedups of 5.2 to 6.7 times, and CUDA came within a few percent of OpenMP without beating it.

1 Introduction

Frequent itemset mining starts from a database of *transactions*, each a set of *items*: the products in one shopping basket, or the attribute values that describe one mushroom. The *support* of an itemset is the number of transactions that contain all of its items. Given a minimum support threshold, the task is to find every itemset whose support is at least the threshold, together with its support. The threshold can be an absolute count or a fraction of the transactions; my program accepts both. Frequent itemsets are the input to association rules: the confidence of a rule $X \Rightarrow Y$ is the support of $X \cup Y$ divided by the support of X.

With n distinct items there are $2^n - 1$ possible itemsets, so no algorithm can count them all. Every practical algorithm uses the *downward closure* property: every subset of a frequent itemset is frequent, so an itemset with an infrequent subset can be skipped. Apriori [1] uses this level by level. It generates candidate k-itemsets from the frequent $(k-1)$-itemsets, prunes them, and scans the whole database to count the survivors. The candidate sets can be very large, and frequent itemsets of length 15 need at least 15 database scans.

FP-Growth [2] avoids both costs. It scans the database exactly twice to build a compact prefix tree, the FP-tree, and then mines the tree recursively through smaller *conditional* trees without generating any candidates: every itemset it emits is known to be frequent.

2 The FP-Growth algorithm

2.1 The FP-tree

An FP-tree is a prefix tree over the transactions with three additions. **Item ranks:** the frequent items are sorted by descending support, and an item's position in that order is its rank. Each transaction is inserted as its frequent items sorted by rank, so the most frequent items sit nearest the root, where the most transactions can share them. **Counts:** each node stores an item and the number of transactions whose sorted item list passes through it. **Header table and node-links:** the header table has one entry per frequent item, with its support and a pointer to the first node that carries the item; each node points to the next node with the same item, so the miner can visit every occurrence of an item without searching.

2.2 Building the tree in two passes

Pass 1 scans the database and counts every item. Items below the threshold are discarded and the rest are sorted to give the ranks. **Pass 2** scans the database again, removes the infrequent items from each transaction, sorts the rest by rank and inserts them: from the root, it follows the child with the next item, creating it (and linking it into the item's node-link chain) if it does not exist, and increments the count of every node on the path. The tree is never larger than the number of frequent-item occurrences and is usually much smaller.

2.3 Recursive mining

Mining goes through the header table from the least to the most frequent item. For an item a with support s, and a *suffix* of items fixed by the enclosing recursion levels (empty at the top):

1. Output $\{a\} \cup$ suffix with support s.
2. Follow a's node-links. Each node's path up to the root is a *prefix path* that occurs as often as the node's count. These weighted prefix paths form the *conditional pattern base* of a: the transactions that contain a, restricted to the items ranked before a.
3. Count the items in the pattern base, drop the infrequent ones, and build an FP-tree from the weighted paths: the *conditional FP-tree* of a.
4. Mine the conditional tree recursively with a added to the suffix.

Each itemset is found exactly once, under its lowest-ranked item, because only items ranked before a appear in a's pattern base. **Single-path optimization:** if a tree is a single path, every non-empty combination of its nodes is frequent, with the count of its deepest node as support (counts never increase down a path). A path of k nodes yields its $2^k - 1$ itemsets directly, without building more trees.

2.4 Complexity

The two passes take time linear in the total transaction length, plus sorting each transaction. Mining has no useful bound in the input size, because the output can be exponential: one path of k frequent items already produces $2^k - 1$ itemsets. The useful measure is the cost per itemset produced. FP-Growth spends work only on itemsets that are actually frequent, while Apriori also counts candidates that turn out infrequent. Section 7.2 shows that on dense data my running time is roughly proportional to the number of itemsets.

3 Implementation

Language and build. The implementation is C++17 in fptree.h/.cpp (the tree), fpgrowth.h/.cpp (input, the two passes, mining, output) and main.cpp (command line). make builds it with g++ -std=c++17 -O2 -DNDEBUG -Wall -Wextra -Wpedantic; make debug builds a second binary with AddressSanitizer for the tests. The edge-case tests are a shell script; the other tests and the benchmarks are Python scripts run in a virtual environment that the Makefile creates.

Node pool with 32-bit indices. All nodes of a tree live in one std::vector<FPNode>, and every link (parent, first child, next sibling, next node-link) is a 32-bit index into it, with -1 for "none". A node has six 4-byte fields, so it takes 24 bytes. I chose this over pointer-linked nodes because a tree is built once and freed all at once (one deallocation, no leaks); a 4-byte index is half the size of a pointer and the nodes are contiguous, which helps the cache; freeing a tree of unique_ptrs recurses along every path and can overflow the stack on long transactions; and index-based nodes map directly onto flat GPU arrays (Section 8.2). During insertion, a hash map keyed by (parent, item rank) finds children in constant time even at a root with thousands of children; it is freed once the tree is complete. Transactions are stored back to back in one array with an array of start offsets (CSR layout), sorted and de-duplicated.

Item ranks and tie-break. Items are ranked by descending support, ties broken by ascending item id, in the initial tree and in every conditional tree. This makes the trees, counters and output deterministic. As Section 4.1 shows, it is also the only reason two of my conditional trees for the textbook example are ordered differently from the book.

Input, output and command line.

fpgrowth <input_file> <min_support> [output_file] [--print-tree] [--unsorted]

The input is the FIMI format: one transaction per line, items as space-separated non-negative integers; blank lines are skipped and repeated items dropped. Each output line is an itemset and its support, such as 1 2 5 (2). By default the itemsets are buffered and written in sorted order so that runs can be compared with diff; --unsorted writes each one as it is found. min_support is a count (3) or a fraction (0.05). A fraction is rounded up to a count, unless a count matches it at the precision it was typed: 0.266667 with 15 transactions gives 4, not 5. --print-tree prints the header table and the initial tree.

Instrumentation. Each run writes one line of key=value pairs to standard error: phase timers read_s, build_s (both passes), mine_s, output_s (sorting and writing buffered output) and total_s; the peak resident memory peak_rss_kb from getrusage; and counters for frequent items, tree_nodes, filtered_occurrences (item occurrences inserted into the tree), itemsets, cond_trees and cond_nodes (conditional trees built and their nodes) and single_paths. With --unsorted, itemsets are formatted while mining, so formatting is part of mine_s.

4 Validation

4.1 The Han and Kamber example

The textbook example [3] has the nine transactions of Table 1 over items I1 to I5, mined with a minimum support count of 2. I encoded I*k* as the integer *k* in data/examples/han.dat.

Table 1: The Han and Kamber example transactions (data/examples/han.dat).

TID	Items	TID	Items	TID	Items
T100	I1, I2, I5	T400	I1, I2, I4	T700	I1, I3
T200	I2, I4	T500	I1, I3	T800	I1, I2, I3, I5
T300	I2, I3	T600	I2, I3	T900	I1, I2, I3

./fpgrowth data/examples/han.dat 2 --print-tree reports the supports I2: 7, I1: 6, I3: 6, I4: 2 and I5: 2, and a tree of 10 nodes built from 23 item occurrences, drawn in Figure 1. The tie-break puts I1 before I3 and I4 before I5, the same header order and supports as Figure 6.7 of [3, p. 258], and the tree has the same nodes and counts as that figure. Only the left-to-right order of siblings and the way the node-links are drawn differ.

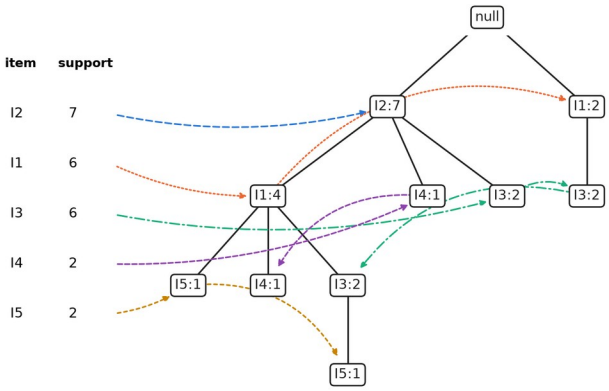


Figure 1: The initial FP-tree of the Han and Kamber example, drawn from the program's --print-tree output by scripts/han_tree.py. Dashed chains are the node-links of each item, in the order the program created the nodes. Notice that every path follows the rank order I2, I1, I3, I4, I5, and that T500 and T700, which start with I1, form their own branch.

Mining walkthrough. The miner works from I5 up to I2 (Table 2).

- **I5:** two nodes of count 1 with prefix paths ⟨I2, I1⟩ and ⟨I2, I1, I3⟩. I3 has support 1 in this base and is dropped. The conditional tree is a single path, so the shortcut emits {I1, I5}, {I2, I5} and {I1, I2, I5}, each with support 2.
- **I4:** prefix paths ⟨I2, I1⟩ and ⟨I2⟩; only I2 reaches 2, giving {I2, I4}: 2.
- **I3:** prefix paths ⟨I2, I1⟩: 2, ⟨I2⟩: 2, ⟨I1⟩: 2, so I1 and I2 both have support 4. The conditional tree has two branches and is mined recursively: I2 in it has the prefix path ⟨I1⟩: 2, giving {I2, I3}: 4 and {I1, I2, I3}: 2, and I1 gives {I1, I3}: 4.
- **I1:** the only prefix path is ⟨I2⟩: 4, giving {I1, I2}: 4.
- **I2:** a child of the root, so it has no pattern base.

Table 2: Mining the example item by item. The single items {I1}: 6, {I2}: 7, {I3}: 6, {I4}: 2 and {I5}: 2 are emitted too.

Item	Conditional pattern base	Conditional FP-tree (mine)	Itemsets generated
I5	{I2, I1: 1}, {I2, I1, I3: 1}	⟨I1:2, I2:2⟩	{I2, I5}: 2, {I1, I5}: 2, {I1, I2, I5}: 2
I4	{I2, I1: 1}, {I2: 1}	⟨I2:2⟩	{I2, I4}: 2
I3	{I2, I1: 2}, {I2: 2}, {I1: 2}	⟨I1:4, I2:2⟩, ⟨I2:2⟩	{I2, I3}: 4, {I1, I3}: 4, {I1, I2, I3}: 2
I1	{I2: 4}	⟨I2:4⟩	{I1, I2}: 4

The program's counters agree: 5 conditional trees with 8 nodes in total (2, 1, 3 and 1 nodes for I5, I4, I3 and I1, plus the 1-node tree of I2 inside I3's tree), and 4 single paths. The output is these 13 itemsets, the 13 frequent itemsets of the book: 1 (6), 1 2 (4), 1 2 3 (2), 1 2 5 (2), 1 3 (4), 1 5 (2), 2 (7), 2 3 (4), 2 4 (2), 2 5 (2), 3 (6), 4 (2), 5 (2).

Difference from the textbook. Table 6.2 of [3, p. 259] lists the same four conditional pattern bases and the same frequent patterns with the same supports as Table 2. The only difference is the item order inside two conditional FP-trees. The book gives I3's conditional tree as ⟨I2:4, I1:2⟩ plus ⟨I1:2⟩, with I2 on top. Mine is ⟨I1:4, I2:2⟩ plus ⟨I2:2⟩, with I1 on top, because I1 and I2 tie at support 4 in I3's pattern base and my rule puts the lower item id first. Both trees hold the same three weighted prefix paths and yield the same itemsets {I2, I3}: 4, {I1, I3}: 4 and

{I1, I2, I3}: 2. The same tie makes my single path for I5 ⟨I1:2, I2:2⟩ where the book lists ⟨I2:2, I1:2⟩, with the same itemsets. I confirmed both trees by writing each pattern base as a small dataset and running --print-tree on it, which applies the same ordering rule.

4.2 Automated tests

make test runs four kinds of tests, which I re-ran for this report (Table 3). **Edge cases:** six hand-written inputs (empty file, single transaction, identical transactions, nothing frequent, everything frequent, and one 12-item transaction that must produce all $2^{12} - 1 = 4095$ subsets). **Brute-force differential test:** each trial generates a random dataset of 5 to 30 transactions over 4 to 10 items, with duplicate items, blank lines and irregular whitespace; a Python reference computes every itemset's exact support by enumerating subsets, and the binary must match it exactly at several absolute and fractional thresholds. **AddressSanitizer:** the edge cases and 200 brute-force trials also run on the ASan build, where any sanitizer report is a failure. **mlxtend cross-check:** tests/compare_mlxtend.py compares itemsets and supports exactly with mlxtend's fpgrowth [5]; make test runs it on the Han example, and I also ran it on all 13 UCI settings of Section 6.6.

Table 3: Test results from re-running make test and tests/compare_mlxtend.py for this report.

Test	Binary	Result
Edge cases	fpgrowth	6 of 6 passed
Edge cases	fpgrowth_debug (ASan)	6 of 6 passed
Brute force, 500 datasets (seed 1)	fpgrowth	3,356 of 3,356 runs passed
Brute force, 200 datasets (seed 2)	fpgrowth_debug (ASan)	1,354 of 1,354 runs passed, 0 sanitizer reports
mlxtend, Han example	fpgrowth	13 itemsets, identical
mlxtend, 13 settings on the 4 UCI datasets	fpgrowth	13 of 13 identical (366 to 468,003 itemsets)

5 Datasets

I used four datasets from the UCI Machine Learning Repository [4]: **Mushroom** [6] (8,124 mushrooms, 22 attributes and an edible/poisonous class), **Chess (King-Rook vs. King-Pawn)** [7] (3,196 end-game positions, 36 board features and a win/no-win class), **Connect-4** [8] (67,557 game positions, one attribute per each of the 42 squares and a win/loss/draw class) and **Online Retail** [9] (invoice lines of a UK online retailer). make data downloads them and converts them to FIMI files, with a .map.tsv file that names every item id for decoding results.

Attribute=value encoding. Each row of the three categorical tables becomes one transaction with one item per attribute=value pair, such as odor=foul. The class is included as an ordinary attribute, so rules that predict it appear among the itemsets. Item ids are assigned in column order with sorted values, so the numbering is deterministic, and missing values (?) are skipped. Every row has

(almost) every attribute, so all Chess transactions have 37 items and all Connect-4 transactions 43.

Why Mushroom has 118 items. The Mushroom documentation lists 127 attribute values, including the two classes. Nine never occur in the data (for example veil-type=universal and stalk-root=rhizomorphs) and get no item id, which leaves 118. The only missing values are 2,480 ? entries in stalk-root; those rows have 22 items instead of 23.

Online Retail cleaning. Each invoice is a transaction of StockCodes. Of the 541,909 raw rows, I removed 9,288 rows of cancelled invoices (numbers starting with C), 1,336 further rows with quantity ≤ 0, and 2,366 further rows that are not products: postage, carriage, fees, manual adjustments, discounts, samples and gift vouchers (POST, DOT, M, C2, D, S, BANK CHARGES, AMAZONFEE, CRUK, PADS, B, GIFT_*). The remaining 528,919 rows form 20,538 invoices over 3,815 products; a product bought twice on one invoice counts once.

Table 4 gives the statistics. *Density*, the average transaction length divided by the number of items, separates the dense Chess (0.4933) from the very sparse Online Retail (0.0066), whose baskets hold 1 to 1,106 items.

Table 4: Dataset statistics after preprocessing.

Dataset	Transactions	Distinct items	Avg len	Min len	Max len	Density	File size
Chess	3,196	75	37.00	37	37	0.4933	331.2 KB
Mushroom	8,124	118	22.69	22	23	0.1923	542.4 KB
Connect-4	67,557	129	43.00	43	43	0.3333	8.8 MB
Online Retail	20,538	3,815	25.24	1	1106	0.0066	2.3 MB

6 Experimental results

6.1 Methodology

All serial measurements were made on an Intel Core i7-14700K (8 performance and 12 efficiency cores, 28 logical CPUs) with 15 GiB of memory, under Ubuntu 26.04 in WSL2 on Windows, with g++ 15.2.0 and -std=c++17 -O2 -DNDEBUG. The serial binary is single-threaded and runs were made one at a time.

- Every reported value is the **median of 5 runs**, because WSL2 and the hybrid P/E-core scheduler add timing noise.
- All runs used **--unsorted with output to /dev/null**, so writing a real file does not distort time or memory. The itemsets are still fully formatted as text, so the **mining time includes formatting every itemset**.
- Every run was capped at a **4 GB address space** and a **120 s timeout**. A support sweep stopped at the first level that took over 60 s or used over 4 GB. Peak memory is the binary's own ru_maxrss.

Table 5: Support sweep summary, from the highest to the lowest completed support level (medians of 5 runs).

Dataset	Levels	Min support	Frequent itemsets	Total time	Peak RSS	Nodes / occurrences	Stopped at
Chess	9	95% –	78 – 99M	1.45 ms –	5.1 – 6.8	0.0018 –	20% (run > 60 s)

Dataset	Levels	Min support	Frequent itemsets	Total time	Peak RSS	Nodes / occurrences	Stopped at
		25%		27.4 s	MB	0.2855	
Mushroom	10	40% – 0.5%	565 – 246M	3.07 ms – 54.5 s	5.9 – 7.1 MB	0.0054 – 0.1454	not reached (lowest 54.5 s)
Connect-4	8	95% – 45%	2.21k – 173M	43.6 ms – 44.1 s	44.7 – 44.7 MB	0.0001 – 0.0034	40% (run > 60 s)
Online Retail	8	5% – 0.2%	33 – 5.94M	9.4 ms – 4.07 s	10.2 – 36.3 MB	0.3486 – 0.9152	0.15% (killed at 120 s)

6.2 Runtime and number of itemsets

Table 5 summarizes the sweep. Runtime (Figure 2) and the number of itemsets (Figure 3) have nearly the same shape on every dataset. Chess goes from 78 itemsets in 1.45 ms at 95% to 99,022,533 itemsets in 27.4 s at 25%; at 20% it found 291,225,060 itemsets in 80.4 s, over the budget. Connect-4 reaches 173,499,179 itemsets in 44.1 s at 45%, and Mushroom 245,818,845 in 54.5 s at 0.5%.

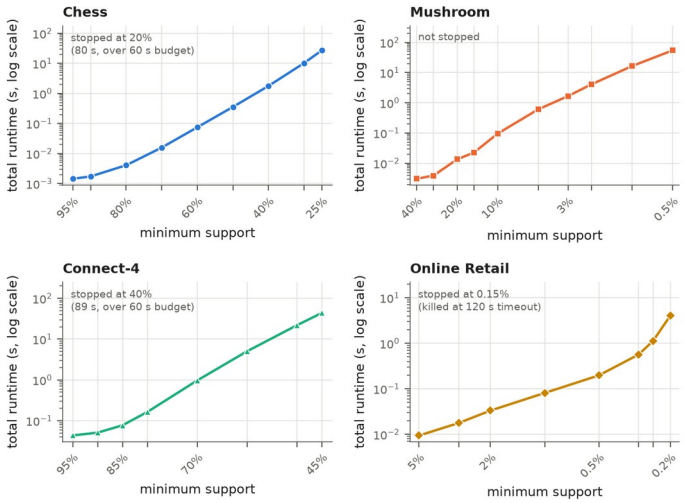


Figure 2: Total runtime (log scale) against minimum support. Notice that the dense datasets give almost straight lines over three to four orders of magnitude, apart from a flatter start at the highest supports, while Online Retail turns sharply upward below 0.25%. The notes say where and why each sweep stopped.

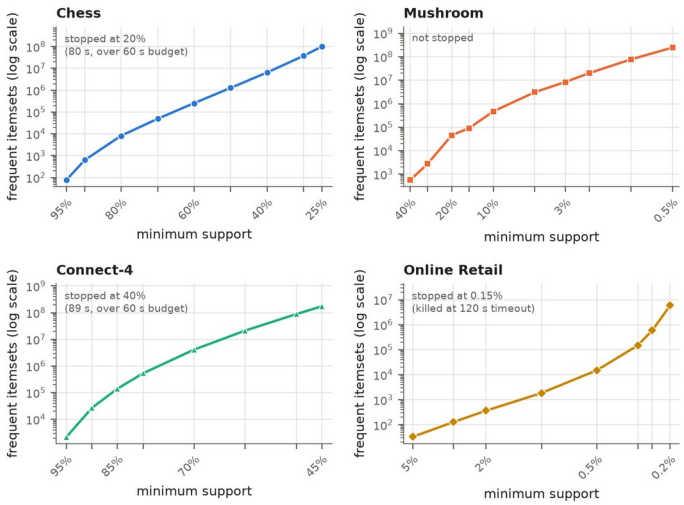


Figure 3: Number of frequent itemsets (log scale) against minimum support. Notice that each panel has the same shape as the runtime panel in Figure 2, except at the highest supports, where reading the input dominates the runtime: runtime follows the size of the output.

6.3 Memory and compression

Peak memory (Figure 4) stays between 5.1 and 7.1 MB on Chess and Mushroom at every level, even with 246 million itemsets. Connect-4 uses 44.7 MB at every level, and Online Retail grows from 10.2 MB at 5% to 36.3 MB at 0.2%.

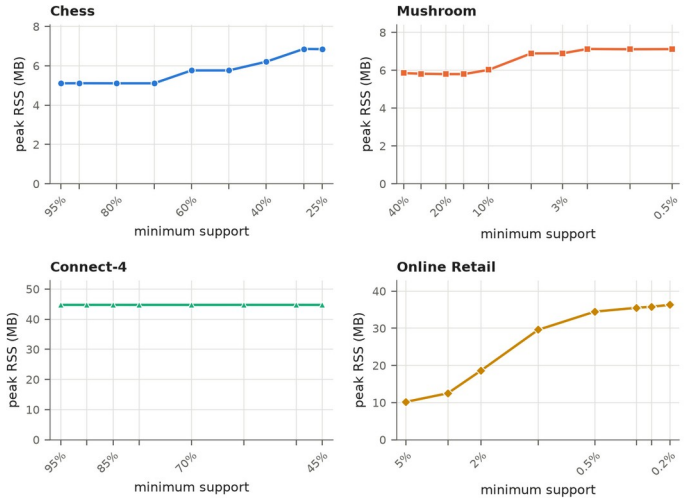


Figure 4: Peak resident memory against minimum support. Notice that memory hardly changes on the dense datasets while the number of itemsets grows by five to six orders of magnitude, because the itemsets are streamed out, not stored.

Figure 5 shows the initial tree's size divided by the number of item occurrences inserted into it (1 would mean no prefix sharing). On Connect-4 it is 0.0001 to 0.0034: at 95%, 1,127,478 occurrences are stored in 140 nodes. On Online Retail it climbs from 0.3486 at 5% to 0.9152 at 0.2%, where the tree has only about 8% fewer nodes than occurrences and is almost one separate path per basket. The ratio rises as the support falls because the newly admitted rarer items appear in few transactions and start new branches instead of extending shared prefixes.

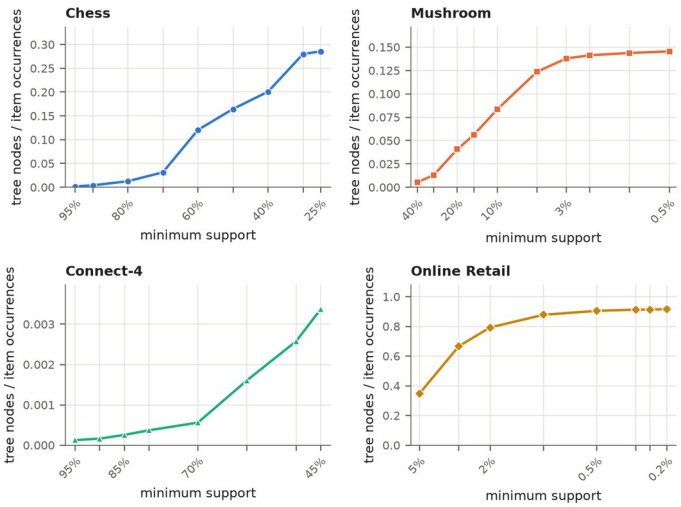


Figure 5: FP-tree nodes per frequent-item occurrence (lower is more sharing). Note the different scales: Connect-4 stays below 0.004 while Online Retail approaches 0.92, and the ratio rises on every dataset as the support falls.

6.4 Phase breakdown and itemset lengths

At a mid-range support per dataset (Figure 6), mining takes 99% of the time on Chess at 50% and about 95% on Mushroom at 10% and Connect-4 at 70%. On Online Retail at 0.5%, building the tree takes 27% and mining 70%, because the tree has 398,233 nodes but only 15,143 itemsets come out. At very high support the balance reverses: on Chess at 95%, reading the file is 75% of a 1.45 ms run.

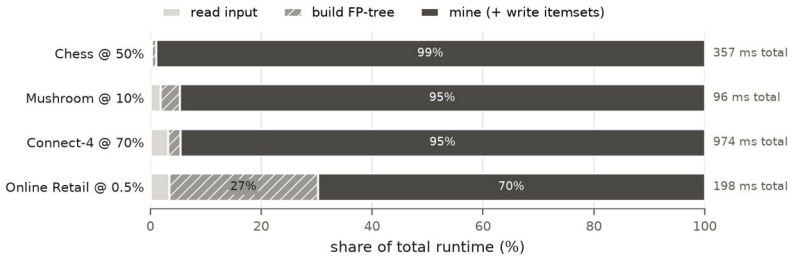


Figure 6: Share of read, build and mine time at one support level per dataset. Notice that the dense datasets spend nearly all their time mining, while Online Retail spends a visible share building its large tree.

Lowering the support shifts the itemsets to longer lengths (Figure 7). On Chess the most common length is 5 at 80% (25.9% of itemsets), 7 at 60% and 9 at 40%, where the longest itemset has 18 items. On Online Retail at 2%, 77.6% of the itemsets are single items and the longest has 3; at 0.2% the most common length is 5 and the longest is 16.

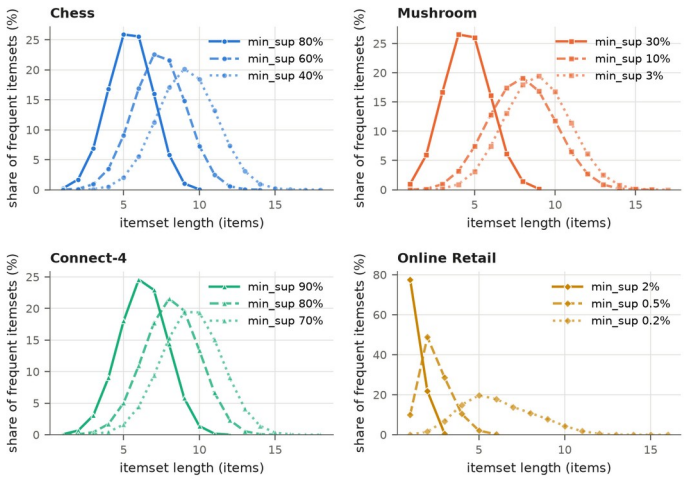


Figure 7: Distribution of itemset lengths at three support levels per dataset. Notice that the peak moves to longer itemsets as the support falls, and that Online Retail changes from mostly single items at 2% to a bell shape like the dense datasets at 0.2%.

6.5 Scalability in the number of transactions

I shuffled Connect-4 and Online Retail once (seed 327), took the first 10, 25, 50, 75 and 100% of the transactions as nested subsets, and mined each at a fixed *relative* support: 70% and 0.5% (Figure 8). On Connect-4, read and build time grow linearly from 5.2 ms to 52.5 ms, but mining stays between 0.834 s and 0.923 s, because the itemset count barely changes (3,797,583 on 10%, 4,129,839 on 100%). On Online Retail, total time grows from 0.041 s to 0.198 s although the itemsets *fall* from 60,331 to 15,143; Section 7.5 explains why.

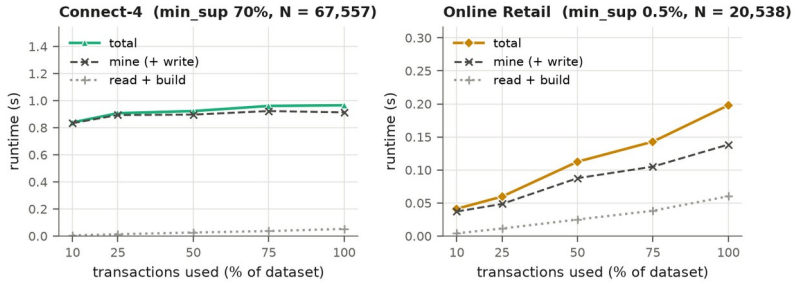


Figure 8: Runtime against the share of transactions used, at fixed relative support. Notice that on Connect-4 only the small read-and-build part grows with the data, while on Online Retail the total grows although larger subsets produce fewer itemsets.

6.6 Comparison with mlxtend

I compared my program with the fpgrowth function of mlxtend [5] 0.25.0 on 13 settings (Figure 9; medians of 3 runs). My time is the total, including reading the file, building the tree, mining and formatting every itemset. **mlxtend's time is only the fpgrowth() call** on an already one-hot-encoded DataFrame; reading and encoding are excluded. Mine was faster in all 13 settings, by 6.3× to 11.3× on the dense datasets and 23.6× to 29.8× on Online Retail, and on Chess at 60% it peaked at 5.8 MB against 373 MB for the mlxtend process. My program is single-

threaded; mlxtend averaged about 1.25 CPU-seconds per wall-second and gave similar times when forced to one thread, so the gap is not a threading effect.

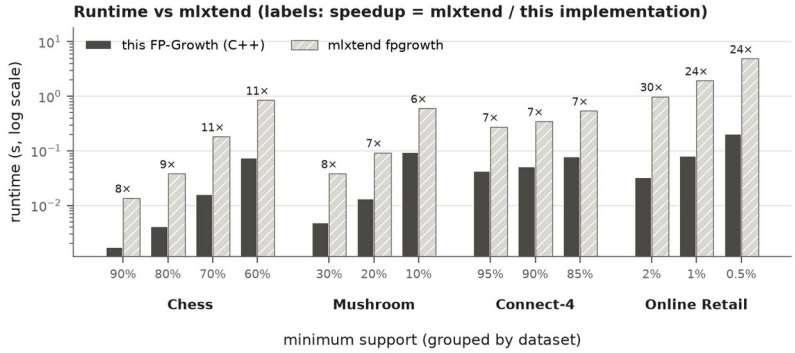


Figure 9: Runtime of this implementation (total, including I/O) and of mlxtend's `fpgrowth()` call (excluding reading and encoding), log scale, labeled with the speedup. Notice that the gap is largest on the sparse Online Retail dataset, although mlxtend is not charged for input processing.

7 Discussion

7.1 Dense versus sparse data

On the dense datasets every transaction contains a large share of all items, so transactions share long prefixes and the tree is tiny: Connect-4's 67,557 transactions fit in 1,081 nodes at 70%. The itemsets, however, are many and long. Online Retail is the opposite: baskets share few prefixes, so the tree is almost as large as the data (398,233 nodes for 440,101 occurrences at 0.5%), but few itemsets are frequent until the threshold is very low. That is why tree building matters only on Online Retail (Figure 6).

7.2 Mining is output-bound

On the dense datasets the mining time per itemset is nearly constant: 275 to 282 ns on Chess from 80% to 25%, 194 to 222 ns on Mushroom from 10% to 0.5%, and 210 to 254 ns on Connect-4 from 90% to 45%, while the itemset count changes by 2.7 (Mushroom) to 4.1 (Chess) orders of magnitude. A perf profile of Connect-4 at 70% put about 75% of all samples in the C++ standard library, formatting itemsets. Conditional trees are a minor cost: they average only 3 to 7 nodes on the dense datasets (cond_nodes / cond_trees).

7.3 Memory is flat because output is streamed

With --unsorted, each itemset is written as soon as it is found, and the recursion holds only the small conditional trees on the current path, so memory is set by the input and the initial tree. Connect-4's 44.7 MB does not change at all while its tree grows from 140 to 7,586 nodes, so it is the cost of reading and storing the 8.8 MB input. Online Retail's memory follows its initial tree, which grows from 14,846 to 454,641 nodes. In the default sorted mode all itemsets are buffered before sorting, so memory would grow with the output; the benchmarks avoid that on purpose.

7.4 The Online Retail explosion below 0.2%

From 5% to 0.5% the itemsets grow only from 33 to 15,143. Going from 0.25% to 0.2% multiplies them by almost ten, from 596,206 to 5,944,725 (runtime 1.12 s to 4.07 s), and at 0.15% the run hit the 120 s timeout. At 0.2% (a minimum count of 41 invoices) the longest frequent itemset has 16 items: 16 products that appear together on at least 41 invoices. By downward closure all $2^{16} - 1 = 65,535$ non-empty subsets of such a group are frequent too, so once long groups like this appear, the output explodes.

7.5 Why Online Retail gets slower on less data

In the scaling experiment, the 10% subset of Online Retail produced 60,331 itemsets and the full dataset 15,143. The cause is the relative support: 0.5% of the subset's 2,054 transactions is a minimum count of only 10 invoices, against 103 on the full data, and far more product groups are bought together 10 times than 103 times. A fixed relative threshold on a small sample of sparse data therefore asks a different question. Runtime still grows with the data because the tree grows from 41,534 to 398,233 nodes. Connect-4 shows no such effect because its frequent items are very common attribute values whose relative frequency barely changes between subsets.

7.6 Example patterns

I decoded some itemsets with the item-name files (scripts/examples.py, output in results/examples.txt).

Mushroom: odor predicts the class. Mining the Mushroom data restricted to the odor and class items shows that every odor except none determines the class. All 2,160 mushrooms with odor=foul are poisonous, and all 400 with odor=almond and all 400 with odor=anise are edible. Odorless mushrooms are mostly edible (3,408 of 3,528, confidence 0.966). Odor alone classifies 4,596 of the 8,124 mushrooms without error.

Online Retail: the teacup rule. The shop sells a "Regency teacup and saucer" in green, pink and roses designs. At 1% support, {GREEN, PINK} appears on 633 invoices and {GREEN, PINK, ROSES} on 542, so the rule {GREEN, PINK} ⇒ {ROSES} has confidence 542 / 633 = 0.856, although only 1,066 of the 20,538 invoices (0.052) contain the roses cup. A customer who buys two colors is very likely to buy the third: a typical recommendation rule.

8 Parallel and GPU implementation

For the bonus I built fpgrowth_omp (OpenMP) and fpgrowth_cuda (CUDA plus OpenMP) from the same sources. Their host code uses the same compiler and flags as the serial binary and only the CUDA kernels go through nvcc, so speed differences come from the parallelization.

8.1 OpenMP version

The top-level loop over the frequent items of the initial tree consists of independent subproblems that share only the read-only initial tree. It runs as

#pragma omp for schedule(dynamic, 1), rarest item first, since rare items have the largest conditional trees and the work per item varies by orders of magnitude. Each thread has its own result sink. In sorted mode the threads' buffers are sorted in parallel and k-way merged, which gives output byte-identical to the serial binary's. With --unsorted, each thread formats into a private 1 MB buffer that is written out in whole lines under a lock, so formatting, the dominant cost, also runs in parallel.

Limit: only the top level is split, so the most expensive single item's subtree bounds the mining time from below. The dense datasets scale almost linearly to 4 threads, then, except Mushroom at 1%, gain little from 8 to 28 (Figure 11). This is most likely that critical path, combined with the CPU's mix of 8 hyper-threaded performance cores and 12 slower efficiency cores. Online Retail, with about 2,000 top-level items, keeps scaling. Splitting deeper recursion levels into tasks would be the next step.

The false-sharing bug. In my first version, each thread's sink and stream buffer were separate small heap allocations whose counters and write pointer change on every itemset. Whether two threads' objects shared a cache line depended on the heap layout, which even the length of the command-line paths changed. With a bad layout, 2 threads on Connect-4 at 50% took 17 to 19 s instead of 10 s. I fixed it by putting each thread's output state into one object aligned to 128 bytes, and re-ran the affected benchmark.

The --unsorted buffering confound. The parallel binaries hand --unsorted output over in 1 MB chunks, while the serial binary writes through std::ofstream's much smaller buffer. That alone makes fpgrowth_omp --threads 1 up to 10% faster than fpgrowth on the dense datasets: Connect-4 at 60% mines in 4.54 s against 5.04 s. With sorted output the two are within 2%. The OpenMP speedups in Figure 11 are therefore relative to the 1-thread OpenMP run.

8.2 CUDA hybrid version

Two facts decided what goes to the GPU: mining is output-bound (Section 7.2), and conditional trees are tiny (on average 3 to 7 nodes on the dense datasets and on average at most about 150 on Online Retail). A CUDA context costs about 0.15 s to create, and one GPU round trip tens of microseconds. So only the regular, data-parallel counting steps go to the GPU:

- **Pass 1, item counting:** one thread per item occurrence rather than per transaction (Online Retail transactions have 1 to 1,106 items, and one long transaction would stall its warp), counting into a shared-memory histogram per block, because global atomics on the dense datasets' only 75 to 129 distinct items would collide constantly.
- **Conditional-support counting:** the first step of building item r's conditional tree walks every node of r up to the root, adding its count to each ancestor's item. For all items at once this is a regular reduction. The tree is flattened into arrays; one block handles one item and one thread one node, adding into a row in shared memory. The result is a lower-triangular matrix holding every frequent 2-itemset and the first step of every conditional tree. The initial tree always goes to the GPU, a conditional tree only if it has at least --gpu-min-nodes nodes (default 4,096).

Table 6: What stays on the CPU in fpgrowth_cuda, and why (from GPU_DESIGN.md).

Part	Why it stays on the CPU
Reading the file	I/O and text parsing

Part	Why it stays on the CPU
Tree insertion (pass 2)	Sequential pointer chasing and hashing with contention near the root; trees of at most 455k nodes build in 30 to 70 ms
Recursion over conditional trees	A few nodes per tree; a kernel launch costs more than the work
Single-path enumeration and output	About 75% of serial mining time is text formatting; the text must end up on the host, and copying gigabytes over PCIe would cost about as much as it saves

Choosing the threshold. With one mining thread (Figure 10), offloading small trees is a large loss: at a threshold of 64, Online Retail at 0.25% counts 10,182 trees on the GPU and mines 59% slower than with the initial tree alone. At 4,096 it counts 323 trees there (the initial tree and 322 conditional trees) and mines in 0.984 s against 1.042 s, a 5.5% gain. On Chess at 40% no setting helps.

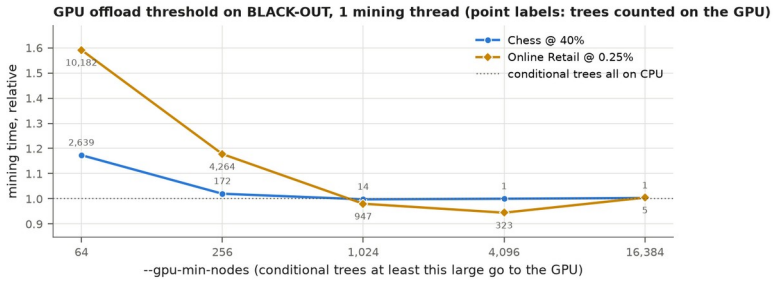


Figure 10: Mining time with one mining thread against `--gpu-min-nodes`*, relative to counting only the initial tree on the GPU (labels: trees counted on the GPU; median of 3). Notice that small thresholds make mining much slower and that 4,096 is the fastest setting, about 5% faster than the CPU on Online Retail.*

8.3 Correctness of the parallel versions

make test-par runs the edge cases at several thread counts, the brute-force test, and a comparison with ./fpgrowth on every dataset at 2 support levels (identical sorted output, the same --unsorted lines, and matching counters). fpgrowth_cuda also runs with --gpu-min-nodes 0 (64 on Online Retail) so the GPU path handles far more trees. My re-run for this report passed every check: 6 of 6 edge cases in each of 7 configurations, all 3,619 brute-force runs (2,683 for fpgrowth_omp, 936 for fpgrowth_cuda) and 32 of 32 dataset comparisons. An earlier run had 3 failures in the fpgrowth_cuda brute-force runs. In each, cudaSetDevice failed with “CUDA-capable device(s) is/are busy or unavailable” and the program exited before reading any data, so no wrong output was produced; replaying the three failed cases gave the correct output. I did not identify the cause.

8.4 Results on the RTX 4070 machine

The parallel benchmarks ran on the same machine with an NVIDIA GeForce RTX 4070 (sm_89, 46 SMs, 12 GiB) and CUDA 12.4, as medians of 5 runs with --unsorted output to /dev/null, each capped at 120 s and 3.8 GiB resident memory (Table 7).

Table 7: Total time in seconds on the RTX 4070 machine, the two hardest levels per dataset (median of 5). OMP n: `fpgrowth_omp` with n threads; CUDA n: `fpgrowth_cuda` with n mining threads. Bold: fastest.

Dataset	Min sup	Itemsets	Serial	OMP 1	OMP 4	OMP 8	OMP 28	CUDA 1	CUDA 28
Chess	30%	37,282,962	10.466	9.485	2.409	1.335	**1.271**	9.673	1.417
Chess	25%	99,022,533	27.771	25.786	6.411	**3.445**	3.724	25.226	3.862
Mushroom	5%	3,082,813	0.615	0.553	0.156	0.101	**0.088**	0.699	0.239
Mushroom	1%	77,322,009	16.596	14.862	3.789	2.038	**1.410**	14.983	1.521
Connect-4	60%	21,252,795	5.102	4.606	1.276	**0.772**	0.828	4.787	0.984
Connect-4	50%	88,316,367	22.061	19.872	5.123	**3.096**	3.183	20.171	3.350
Online Retail	0.25%	596,206	1.137	1.114	0.340	0.209	**0.154**	1.215	0.370
Online Retail	0.2%	5,944,725	4.084	3.927	1.064	0.580	**0.354**	4.051	0.576

OpenMP is the win. The best speedups over the serial binary are 8.2× (Chess 30%), 11.8× (Mushroom 1%), 7.1× (Connect-4 50%) and 11.5× (Online Retail 0.2%). Above about 0.5 s of runtime, scaling is close to linear up to 4 threads (3.3 to 4.0× over the 1-thread run). Below about 0.1 s, thread start-up and the serial read and build phases dominate, and 28 threads are slower than 8.

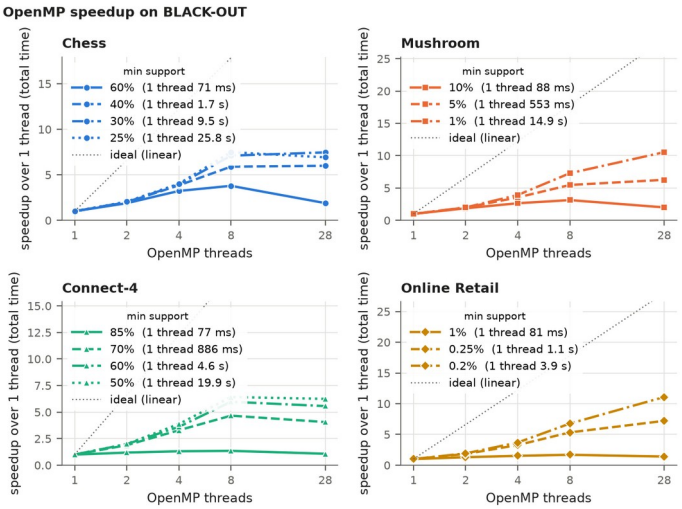


Figure 11: OpenMP speedup in total time over the 1-thread OpenMP run. Notice the near-linear scaling up to 4 threads on all but the shortest runs, the flattening between 8 and 28 threads on the dense datasets, and that the two hardest Online Retail levels and the hardest Mushroom level keep scaling to 28 threads.

CUDA never beats OpenMP on this machine (Figure 12). On the larger settings, CUDA with 28 threads is 0.11 to 0.22 s slower than OpenMP with 28, about the cost of creating the CUDA context (0.14 to 0.22 s), and on the smallest setting of each dataset that start-up alone makes the CUDA binary 2 to 3.8 times

slower than serial. The GPU work itself is small: on the dense datasets the kernels take 1.5 to 4.3 ms and transfers at most 1.2 ms per run. Only Online Retail at 0.25% and 0.2% (322 and 469 conditional trees) and Chess at 30% and 25% (5 and 8) send conditional trees to the GPU, and with 28 threads sharing it, Online Retail at 0.2% takes 0.576 s against 0.354 s for OpenMP alone.

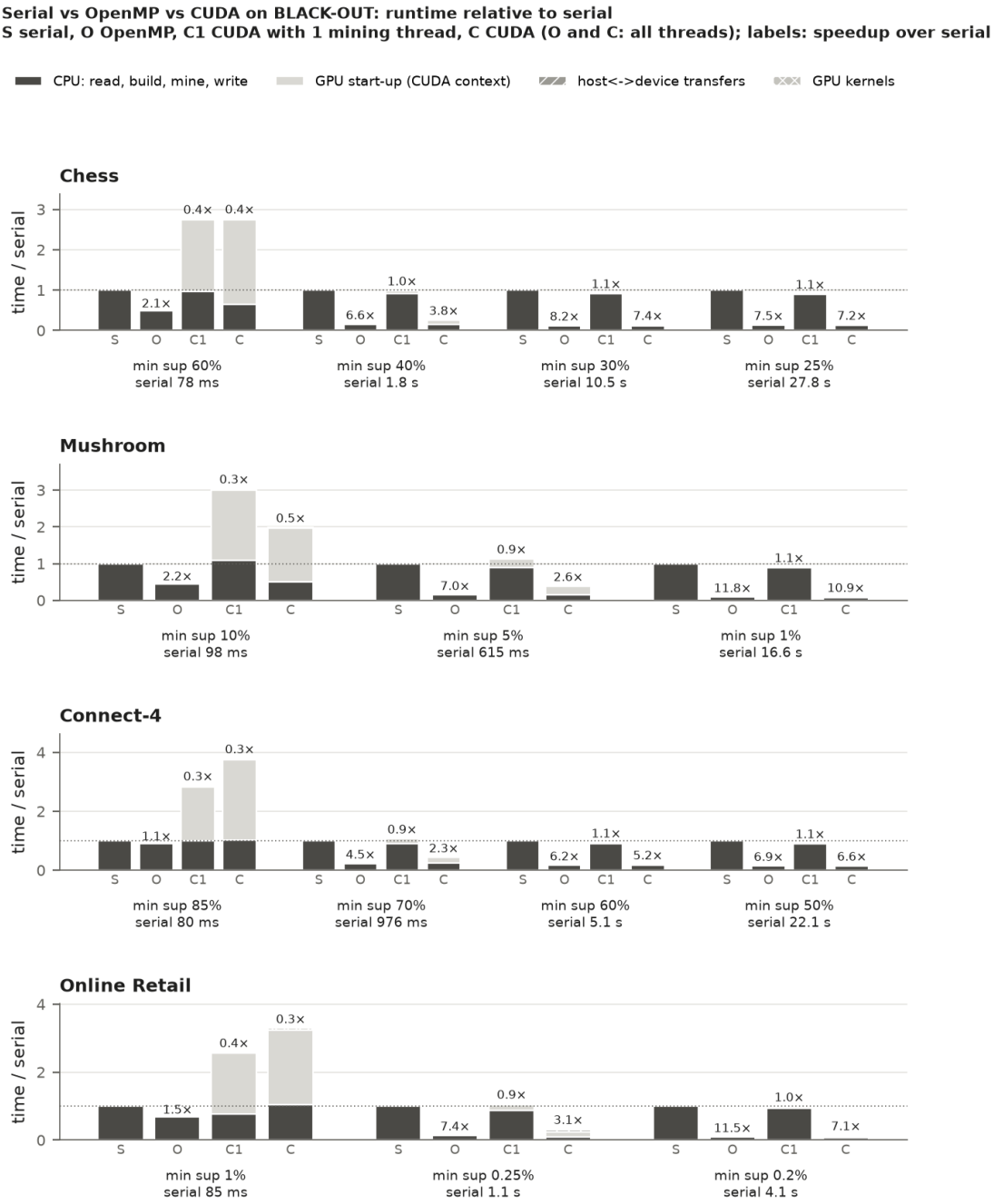


Figure 12: Total time relative to serial for serial (S), OpenMP with all threads (O), and CUDA with 1 (C1) or all (C) mining threads; CUDA bars are split into GPU start-up, transfers, kernels and CPU work. Notice that the light start-up segment is the main difference between O and C, and that kernels and transfers are barely visible, as thin segments on top of the CUDA bars for Online Retail at 1% and 0.25%.

8.5 Kernel level: where the GPU does win

To check whether the kernel itself is fast, fpgrowth_cuda --micro-count 5 times only the step it replaces, the conditional supports of every item of the *initial* tree, on one CPU thread and on the GPU, in one process after the CUDA context exists and after a warm-up, and checks that both results are equal (Figure 13).

On trees above a few thousand nodes the GPU wins clearly: 9 to 110 times for the kernel alone and 4 to 38 times with the copies, mostly growing with tree size (the largest tree gains 83 and 32 times, less than the 297k-node one). On the largest tree (Online Retail 0.2%, 454,641 nodes) the CPU needs 73.5 ms, the kernel 0.88 ms, the kernel with transfers 2.3 ms, and the whole GPU path 9.0 ms. The GPU wins because the work is thousands of independent upward walks with all atomics in shared memory. On the 411-node Connect-4 tree the launch latency (about 25 µs) exceeds the CPU's 8 µs. Sub-millisecond kernel times vary between runs: the Online Retail 0.2% kernel measured 0.56 ms in an earlier run.

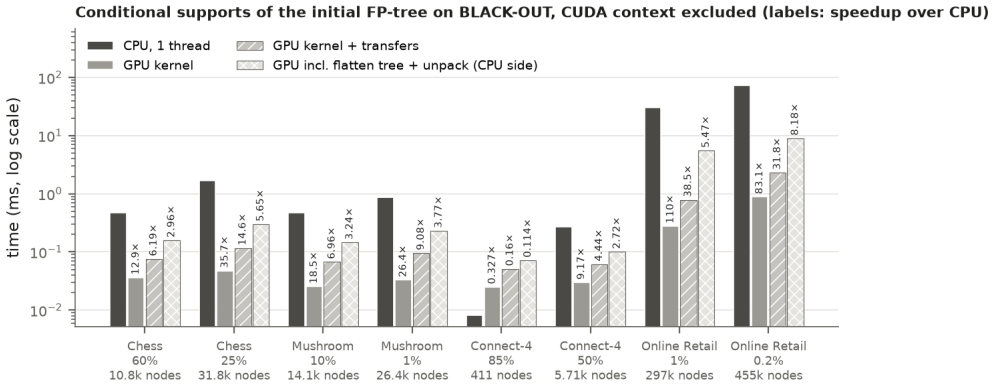


Figure 13: Conditional supports of the initial tree: one CPU thread against the GPU kernel, kernel plus transfers, and the whole GPU path including CPU-side flattening and unpacking (log scale; labels: speedup over the CPU). Notice that the kernel is 9 to 110 times faster on trees above a few thousand nodes, but the CPU-side preparation takes most of that advantage away on the large Online Retail trees.

Four reasons keep this from becoming an end-to-end speedup:

1. **The step is a tiny part of the run.** Its largest instance, 73.5 ms on Online Retail at 0.2%, is 1.8% of the 4.08 s serial run; on Chess at 25% it is 1.7 ms of 27.8 s.
2. **Fixed costs exceed the saving.** The CUDA context costs 0.14 to 0.22 s per run, more than the best saving (73.5 ms down to 9.0 ms). On the large trees, flattening 455k nodes and unpacking the 2.7-million-entry matrix on the CPU take about 6.9 ms, three times the copies and kernel together.
3. **Against OpenMP the CPU work is not serial.** In fpgrowth_omp each item's counting runs inside its own task, spread over all threads. The GPU call for the initial tree runs once, before the parallel loop, on the critical path: 9 ms of wall time, while 73.5 ms of CPU work spread over 28 threads costs only a few milliseconds.
4. **Deeper trees are too small.** The remaining counting happens in conditional trees averaging 3 to 7 nodes on the dense datasets and on average at most about 150 on Online Retail, where the GPU loses even on the kernel alone, as the 411-node tree shows.

The GPU kernel is fast, but it accelerates a small fraction of FP-Growth's runtime on these datasets, which is dominated by irregular, output-heavy CPU work. A multi-core CPU is the better parallel target here. The GPU would matter more with

much larger initial trees, from many more transactions or items at a support where conditional trees stay large.

8.6 Results on the Jetson Orin Nano

The lab results come from an NVIDIA Jetson Orin Nano (board CAD02 in room 281; its hostname is localhost, so its result files are named jetson-orin). make test-par passed on this board for both fpgrowth_omp and fpgrowth_cuda: all edge cases, all 3,619 brute-force runs and 32 of 32 dataset comparisons. A second Jetson board (CAD06) gave nearly identical timings on the runs it completed, for example 7.013 s against 7.012 s for serial Chess at 40%.

The Jetson is a much smaller machine (Table 8): six Arm cores instead of 28 logical CPUs, and a GPU with 8 SMs instead of 46 that has no memory of its own but shares the board's 7.4 GiB with the CPU. It ran in its 25 W power mode. The benchmarks are the same as on the RTX 4070 machine, as medians of 3 runs (5 for the kernel micro-benchmark), each capped at 120 s and 2.9 GiB resident memory. The script picks the support levels from the serial runtimes in results/bench.csv. The Jetson rows have 3 runs each and cover only the first three runtime targets, which is what the script's --quick mode produces, so Chess and Connect-4 have three levels instead of four. All 12 levels were also run on the RTX 4070 machine. On these 12 levels the serial binary is 2.9 to 4.2 times slower on the Jetson.

Table 8: The two machines that ran the parallel benchmarks (from the gpu_info files).

	RTX 4070 machine	Jetson Orin Nano (CAD02)
CPU	Intel Core i7-14700K, 28 logical CPUs	6 Arm Cortex-A78AE cores, 1 thread each, up to 1,728 MHz
Memory	15.6 GiB	7.4 GiB, shared by CPU and GPU
GPU	GeForce RTX 4070, sm_89, 46 SMs, 36 MiB L2	Orin (integrated), sm_87, 8 SMs, 4 MiB L2
GPU memory	12.0 GiB, separate (discrete card)	the same 7.4 GiB as the CPU
Power	not recorded	25 W mode (nvpmodel)
Software	Ubuntu 26.04 under WSL2, g++ 15.2, CUDA 12.4	Ubuntu 24.04, g++ 13.3, CUDA 13.2
Runs per median, memory cap	5, 3.8 GiB	3, 2.9 GiB

Table 9: Total time in seconds on the Jetson Orin Nano (median of 3). OMP n: fpgrowth_omp with n threads; CUDA n: fpgrowth_cuda with n mining threads. Bold: fastest.

Dataset	Min sup	Itemsets	Serial	OMP 1	OMP 2	OMP 4	OMP 6	CUDA 1	CUDA 6
Chess	60%	254,944	0.283	0.260	0.138	0.077	**0.063**	0.325	0.123
Chess	40%	6,472,980	7.012	6.289	3.182	1.607	**1.231**	6.446	1.302
Chess	30%	37,282,962	40.902	36.454	18.323	9.265	**6.718**	36.966	6.744
Mushroom	10%	468,003	0.385	0.337	0.178	0.121	**0.107**	0.399	0.170
Mushroom	5%	3,082,813	2.515	2.187	1.101	0.615	**0.454**	2.255	0.518

Dataset	Min sup	Itemsets	Serial	OMP 1	OMP 2	OMP 4	OMP 6	CUDA 1	CUDA 6
Mushroom	1%	77,322,009	68.925	59.640	29.862	15.072	**10.223**	59.938	10.345
Connect-4	85%	142,271	0.253	0.240	0.191	0.165	**0.160**	0.301	0.217
Connect-4	70%	4,129,839	3.887	3.410	1.781	0.996	**0.711**	3.490	0.776
Connect-4	60%	21,252,795	20.827	18.117	9.147	4.713	**3.361**	18.242	3.403
Online Retail	1%	1,860	0.354	0.356	0.293	0.254	**0.247**	0.355	0.326
Online Retail	0.25%	596,206	3.354	3.316	1.896	1.154	**0.903**	3.195	0.965
Online Retail	0.2%	5,944,725	12.440	11.979	6.338	3.374	**2.400**	11.951	2.462

OpenMP scales well on six cores (Figure 14). On the dense datasets, wherever one thread takes more than 2 s, 4 threads are 3.4 to 4.0 times faster than one and 6 threads 4.8 to 5.8 times. Unlike on the RTX 4070 machine, the curves have not flattened at the last step: going from 4 to 6 threads still gains 1.31 to 1.47 times on these runs (1.5 would be linear). The best speedups over the serial binary are 6.1× (Chess 30%), 6.7× (Mushroom 1%), 6.2× (Connect-4 60%) and 5.2× (Online Retail 0.2%). They can exceed 6 because of the --unsorted buffering confound described in Section 8.1: on the Jetson the 1-thread OpenMP run is up to 13.5% faster than the serial binary. Connect-4 at 85% and Online Retail at 1% (0.24 and 0.36 s on one thread) gain only 1.5 and 1.4 times from 6 threads, because reading and building, which are serial, take 0.135 and 0.220 s of that time. In absolute terms, the Jetson's best OpenMP time on the hardest level of each dataset is 4.4 to 7.3 times the RTX 4070 machine's best time at the same level.

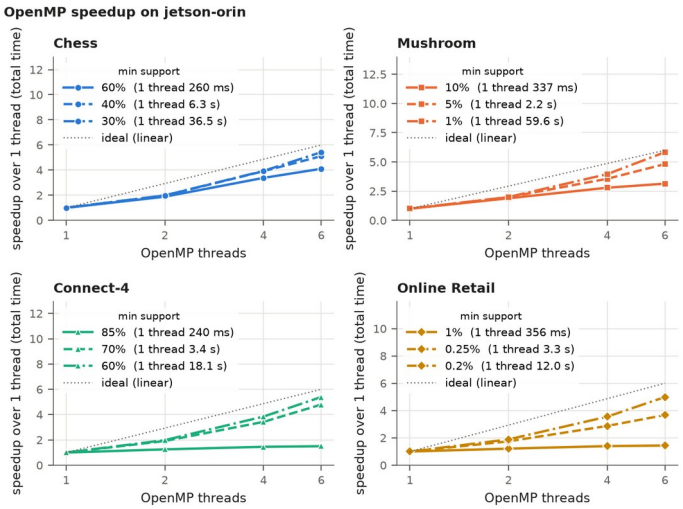


Figure 14: OpenMP speedup in total time over the 1-thread OpenMP run on the Jetson Orin Nano (6 cores). Notice that every curve still rises from 4 to 6 threads, and that Connect-4 at 85% and Online Retail at 1% stay near 1.5, because the serial read and build phases take more than half of their 1-thread time.

CUDA comes close to OpenMP but still does not beat it (Figure 15). With all six threads, fpgrowth_cuda is slower than fpgrowth_omp at every level, by 0.026 to

0.122 s; on the hardest level of each dataset this is only 0.4% to 2.6%, against 0.11 to 0.23 s on the RTX 4070 machine. On the smallest level of each dataset CUDA with 6 threads takes 1.3 to 1.9 times as long as OpenMP, but it is still faster than the serial binary at every level; on the RTX 4070 machine it was 2 to 3.8 times slower than serial at these smallest levels. With one mining thread, CUDA beats OpenMP on Online Retail at 0.25% (3.195 s against 3.316 s, 3.6% faster), the level where 322 conditional trees go to the GPU, and matches it at 0.2% (11.951 s against 11.979 s).

GPU start-up is about a third as expensive. Creating the CUDA context (gpu_init_s) takes 0.054 to 0.061 s on the Jetson in all 24 CUDA runs, against 0.139 to 0.218 s on the RTX 4070 machine, while the CPU work around it is 2.9 to 4.2 times slower. The start-up cost is therefore clearly visible only on the smallest level of each dataset. The two machines differ in GPU type, CUDA version and operating system (the RTX 4070 machine runs under WSL2), and I did not isolate which of these causes the difference.

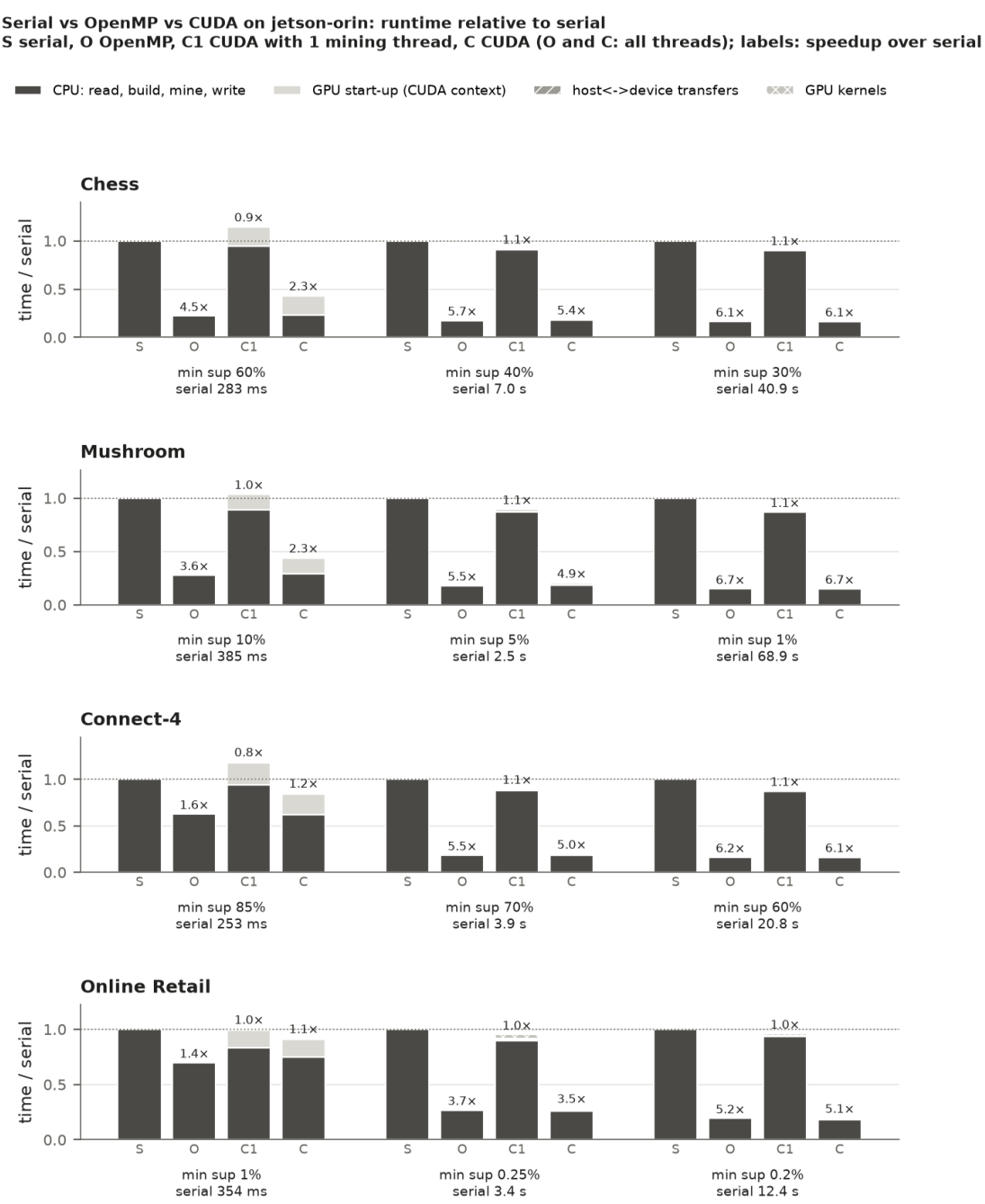


Figure 15: Total time relative to serial on the Jetson Orin Nano for serial (S), OpenMP with 6 threads (O), and CUDA with 1 (C1) or 6 (C) mining threads, split as in Figure 12. Notice that the light start-up segment is clearly visible only at the smallest level of each dataset, and that C is within 3% of O at the hardest level of each dataset.

The offload threshold matters less on the Jetson (Figure 16). With one mining thread, a threshold of 64 makes mining only 1.1% slower on Online Retail at 0.25% and 2.3% slower on Chess at 40%, against 59% and 17% on the RTX 4070 machine. The fastest setting on Online Retail is 1,024 (947 trees on the GPU): 2.644 s against 2.879 s with only the initial tree on the GPU, 8.2% faster. The default of 4,096 gains 5.1% (5.5% on the RTX 4070 machine, where 1,024 gains

1.9%). On Chess at 40% every threshold from 1,024 up is within 0.4% of counting only the initial tree on the GPU, on both machines. At a threshold of 64 on Online Retail, flattening, copies and kernels add up to 0.437 s on the Jetson against 0.625 s on the RTX 4070 machine, while mining with the initial tree alone on the GPU takes 2.8 times as long (2.879 s against 1.042 s), so offloading small trees costs a much smaller share of the run.

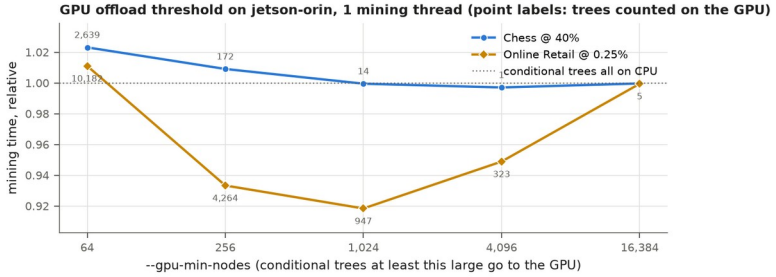


Figure 16: Mining time with one mining thread against `--gpu-min-nodes` *on the Jetson Orin Nano, relative to counting only the initial tree on the GPU (labels: trees counted on the GPU; median of 3). Notice that even a threshold of 64 costs at most 2.3%, and that 1,024 is the fastest setting on Online Retail, about 8% faster.*

At kernel level the GPU still wins, by less (Figure 17). On trees above a few thousand nodes the kernel alone is 2.4 to 52 times faster than one CPU thread (9 to 110 times on the RTX 4070). On the largest tree (Online Retail 0.2%) the CPU needs 233.6 ms, the kernel 6.86 ms (34×), the kernel with transfers 7.55 ms (31×) and the whole GPU path 52.8 ms (4.4×, against 8.2× on the RTX 4070 machine). On the 411-node Connect-4 tree the Jetson's kernel (17.7 µs) is faster than its CPU (26.8 µs), where the RTX 4070 lost (25.0 µs against 8.2 µs), but with the copies it loses too (34.9 µs). On Chess and Mushroom the kernel takes 0.43 to 0.50 ms for trees of 10.8k to 31.8k nodes, much more relative to tree size than the 0.045 ms of the 5.7k-node Connect-4 tree at 50%. I did not investigate the cause.

Transfers cost less, but not nothing. fpgrowth_cuda copies with cudaMemcpyAsync on both machines; on the Jetson the source and destination are in the same memory, not in a separate GPU memory. The copies add 5% to 12% to the kernel time on Chess, Mushroom and Online Retail, against 104% to 191% on the RTX 4070 machine (all trees), and on the largest tree take 0.69 ms against 1.43 ms. They are not free, though: on the Connect-4 trees they add 50% and 97%, and in the full runs the pass-1 copy of the Connect-4 transactions takes 1.7 ms on the Jetson against 1.0 ms on the RTX 4070 machine.

CPU-side preparation dominates the GPU path even more. Flattening the tree and unpacking the result on the CPU take 45.2 ms of the 52.8 ms GPU path on Online Retail at 0.2% (86%) and 92% at 1%, against 77% and 81% on the RTX 4070 machine (6.9 of 9.0 ms at 0.2%). On Chess at 25% and Mushroom at 1% the share is 44.5% to 48.9% on both machines. On the Jetson this preparation, not the kernel, is the largest part of the GPU path on the large Online Retail trees. As on the RTX 4070 machine, the whole step is a small part of the run: 233.6 ms of the 12.44 s serial run on Online Retail at 0.2% (1.9%).

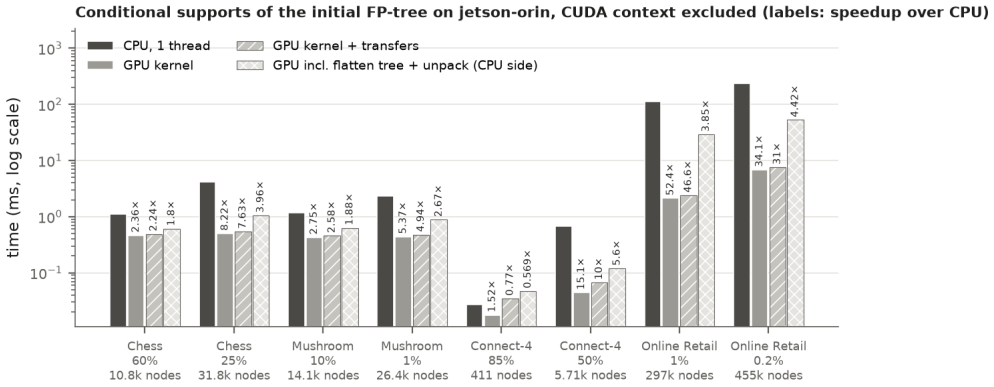


Figure 17: Conditional supports of the initial tree on the Jetson Orin Nano: one CPU thread against the GPU kernel, kernel plus transfers, and the whole GPU path including CPU-side flattening and unpacking (log scale; labels: speedup over the CPU). Notice that the transfer bars are barely taller than the kernel bars except on Connect-4, and that the CPU-side preparation takes most of the GPU path on Online Retail.

The Jetson confirms the conclusion from the RTX 4070 machine: OpenMP gives the speedup, at best 5.2 to 6.7 times on six cores, and the GPU does not add to it. Mostly because of the smaller start-up cost, CUDA comes within 0.4% to 2.6% of OpenMP on the hardest levels, but the work the GPU can take over is still a few percent of the run.

9 Conclusion

I implemented FP-Growth in C++17 with an index-based node pool, deterministic item ranks, and built-in timers and counters. It reproduces the Han and Kamber tree and all 13 itemsets (two conditional trees differ only in order, because of my tie-break), and it passed 3,356 brute-force runs, 1,354 more under AddressSanitizer, six edge cases and exact mlxtend comparisons on 14 settings.

The experiments show that FP-Growth’s cost follows its output. On dense data the tree compresses the input by up to four orders of magnitude and memory stays flat (5.1 to 7.1 MB on Chess and Mushroom, 44.7 MB on Connect-4), while the runtime grows with the number of itemsets at a nearly constant 194 to 282 ns each. On the sparse Online Retail data the tree barely compresses and the output explodes below 0.2% support. My implementation was 6.3 to 29.8 times faster than mlxtend’s fpgrowth() call. OpenMP over the top-level items gave speedups of 7.1 to 11.8 times on 28 logical CPUs. The CUDA kernel is up to 110 times faster than the CPU for its step, but gives no end-to-end gain because that step is a few percent of the runtime. On the 6-core Jetson Orin Nano, OpenMP gave best speedups of 5.2 to 6.7 times, and with a cheaper GPU start-up (0.054 to 0.061 s) and mostly cheaper copies CUDA came within 0.4% to 2.6% of OpenMP on the hardest levels without beating it, so the conclusion is the same. Splitting deeper recursion levels across threads would be the next improvement.

References

[1] R. Agrawal and R. Srikant, “Fast algorithms for mining association rules,” in *Proc. 20th Int. Conf. on Very Large Data Bases (VLDB)*, 1994, pp. 487–499.

[2] J. Han, J. Pei, and Y. Yin, “Mining frequent patterns without candidate generation,” in *Proc. ACM SIGMOD Int. Conf. on Management of Data*, Dallas, TX, 2000, pp. 1–12.

[3] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Morgan Kaufmann, 2012.

[4] Markelle Kelly, Rachel Longjohn, Kolby Nottingham, The UCI Machine Learning Repository, https://archive.ics.uci.edu

[5] S. Raschka, “MLxtend: Providing machine learning and data science utilities and extensions to Python’s scientific computing stack,” *Journal of Open Source Software*, vol. 3, no. 24, p. 638, 2018. Version 0.25.0 used.

[6] “Mushroom,” UCI Machine Learning Repository, 1981. [Online]. Available: https://doi.org/10.24432/C5959T. Records drawn from *The Audubon Society Field Guide to North American Mushrooms*, G. H. Lincoff (Pres.), New York: Alfred A. Knopf, 1981, pp. 500–525, as given in the dataset’s agaricus-lepiota.names file.

[7] A. Shapiro. “Chess (King-Rook vs. King-Pawn),” UCI Machine Learning Repository, 1983. [Online]. Available: https://doi.org/10.24432/C5DK5C.

[8] J. Tromp. “Connect-4,” UCI Machine Learning Repository, 1995. [Online]. Available: https://doi.org/10.24432/C59P43.

[9] D. Chen. “Online Retail,” UCI Machine Learning Repository, 2015. [Online]. Available: https://doi.org/10.24432/C5BW33.

Appendix: Building, running and repository layout

Build requirements, instructions for every program and make target, the repository layout and the steps to reproduce every result in this report are in README.md at the top level of the submitted zip file.